Dynamic Memory Management

CS 1622

Jarrett Billingsley

Class Announcements

- we're gonna talk about some stuff a bit abstractly today
  - but I think it's a really important topic
  - dynamic memory management has a **H U G E** impact on language and compiler design
  - and it will also be important to know about this for when we start talking about code generation and the runtime library!

Allocation, Deallocation, Lifetime, and Ownership

Making room

- every value in your program **takes up space in memory.**
- **allocation** sets aside a piece of memory as a value's "home."

```
let g = 10;
let h = 20;
fn main() {
  g = g + h;
}
```

global variables are the easiest to allocate, because **the compiler knows** how many globals there are.

it does **static allocation:** it gives each global a unique location, which is encoded in the output machine code.

static allocation is also used for certain kinds of constants, like `"quoted string literals"`.

But that's boring

- what about **locals?** they "appear" and "disappear" with function calls.
- that means we have to do **dynamic allocation:** we come up with the locations of variables at **runtime.**

```
fn r(x: int) {
 if x > 1 {
  return x + r(x-1);
 } else {
  return x;
 }
}
```

this is easier than it sounds at first, because every function has a **fixed number of variables.**

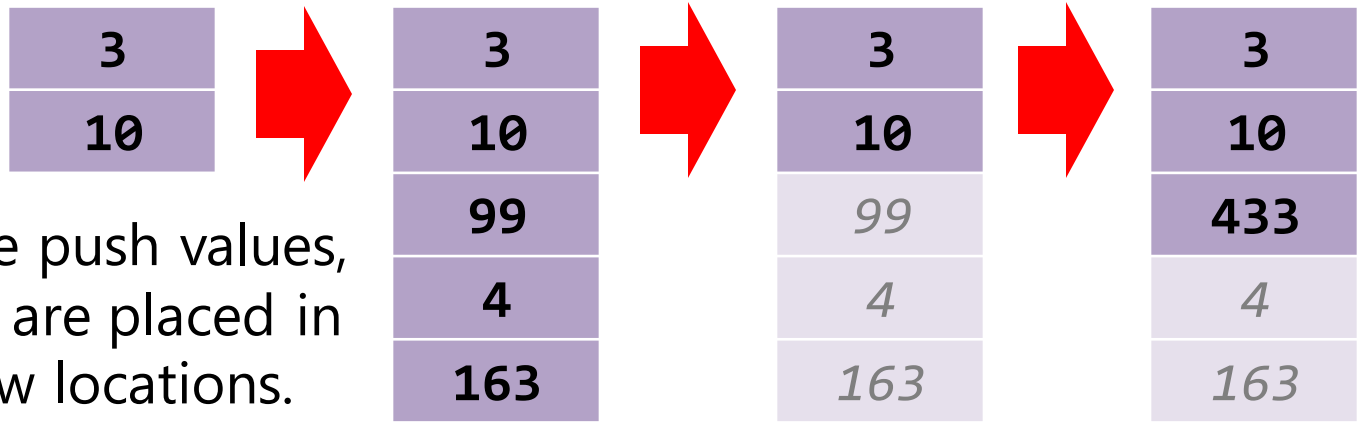
because of how function calls work, the variables can be allocated using a **stack.**

so, each local variable gets a **statically-determined location** within the stack frame, *relative to the sp*.

Deallocation

- memory is a **finite resource.** we have to **reuse** it when we can.
- **deallocation** marks a previously-used location as ready for reuse.

stacks are natural recyclers.



as we push values, they are placed in new locations.

when they're popped, the *values* are deallocated, but the *locations* stick around…

and can be easily reused by the next push.

Lifetime and Ownership

- a value's **lifetime** is the span of time from allocation to deallocation.
- its **owner** is **what decides when it's safe to deallocate it.**

```
let g = 10;

fn main() {
  let x = g + 5;
  println_i(x);
}
```

global variables are owned by **the program:** they can't be deallocated until the program ends.

local variables are owned by **the enclosing function:** they can be deallocated when it returns.

...but this isn't the whole story, is it?

Indirection

time check < 20 min

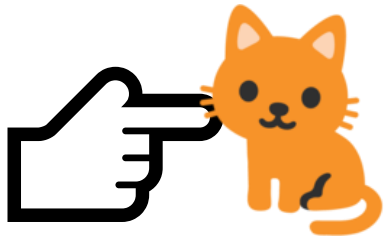
Indirection, the concept

- **indirection** means using a symbol to **stand in** for something, instead of using that thing directly.
  - yes, this is really abstract, but it's an abstract concept!



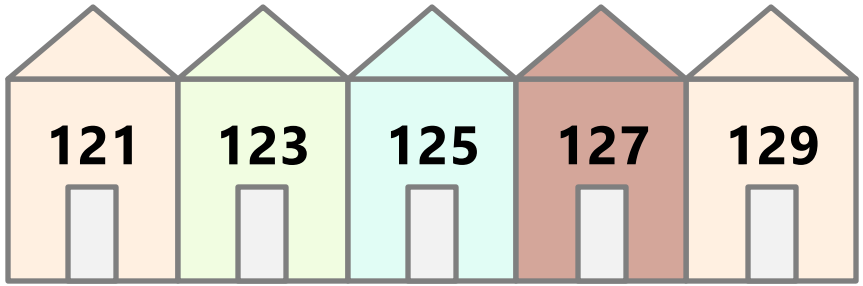
linguistic symbols – spoken or written – all stand in for something else.





if I point at a cat, **my finger** is the symbol which refers to the cat.

if I want to refer to houses on my street, I can refer to them **by number.**



It works for houses, and it works for objects

- if I have a **program** which deals with very large objects (kB-MB?),
  - I can put them in an **array** and access them by their **index.**

64kB	64kB	64kB	64kB	64kB	64kB	64kB
0	*1*	*2*	*3*	*4*	*5*	*6*

the number 3 is small enough to fit into a register.

of course, memory *itself* is an **array** (of bytes), so we can refer to anything *in* memory by its index: its **memory address.**

01	00	00	80	F8	FF	FF
8000	*8001*	*8002*	*8003*	*8004*	*8005*	*8006*

Pointers and references

- programs manipulate **values,** like ints, strings, objects, etc.
- a **pointer** is a value which is the **memory address of another value.**
  - **"reference"** is another name for a pointer, with a "safer" connotation.

```
let x = 1;
let y = 5;
let r = &x;
```



we say, "r **points to** x." r is the pointer, and x is its **referent.**

when you run this code, the variables might look something like this in memory.

if I now write this:

```
let z = *r;
```

the asterisk **follows the arrow from r to x,** and loads the value stored there. so z will be...?

Dereferencing

- **dereferencing** means "accessing the referent of a reference."
- **it does not mean "to deallocate a piece of memory."**

```
A a = new A();
a.x = 10;
a.method();
arr[i] = a;
```

in Java, the dot and square bracket operators perform dereferencing, to access values and methods within an object or array.

```
let mut x = 1;
let r = &mut x;
let z = *r;
*r = 10;
```

Rust has those, as well as the prefix asterisk operator, which can **get** or **set** the value at the other end of the reference.

The necessity of indirection

- when you write a program, you have a **fixed number** of variables...
  - but the size of most data structures is **dynamic.**
  - you don't know how many values there will be until *runtime*.
- so, we have to **dynamically allocate memory:**

```
int[] a = new int[100]; // gimme 100 new variables
```

- but we won't know the array's *address* until runtime!
  - so the only way to access it is **indirectly.**
- the variable above is a pointer to an value on the **heap,** the area of memory where dynamic allocation happens.
  - from now on, I'll use the term **object** to refer to **heap-allocated values,** but not necessarily in the "object-oriented" sense.

The power of the heap

- the heap lets us create data structures of *any* practical size at runtime.
- the **lifetime** of heap-allocated values (**objects**) is also dynamic.

```
fn f(): Cat {
  return new Cat();
}

fn main() {
  let c = f();
  c.meow();
}
```

an object can outlive the function which allocated it!

but this presents a problem:
when does the lifetime of an object end?

(who's the owner??)

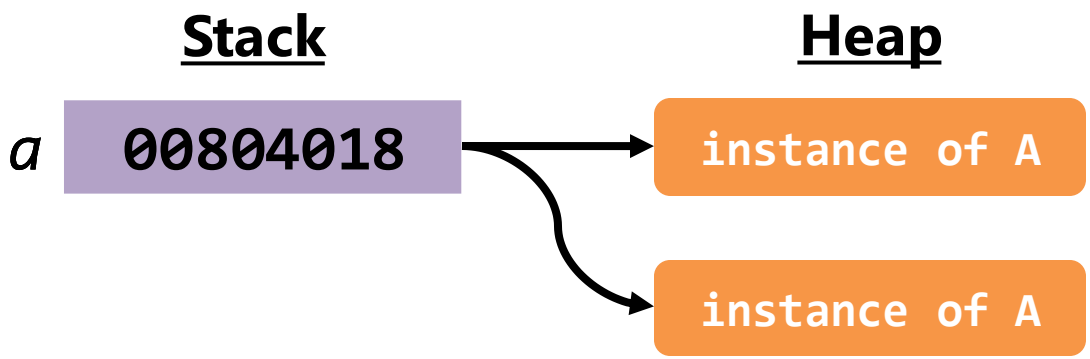
Heap memory management

time check < 40 min

Memory safety (slightly animated)

- a program is **memory-safe** if, when you follow an arrow:
  - there is exactly **one possible value** on the other end; and
  - that value is guaranteed to be **alive.**

```
fn main() {
 let a = new A();
 a.method();
 a = new A();
 a.method();
}
```



at all times, **a** is pointing to a **live** object, so calling a method on it is safe.

for memory safety to hold, as long as *at least one arrow* is pointing to an object, **it must not be deallocated.**

but that sounds hard to prove…

So let's not prove it!

- one way to ensure arrows **always** point to live objects is...
  - you ***never* deallocate them!**
- what?? didn't we say memory was finite, and we have to reuse it?
  - well, yeah, but not *every* program needs to reuse memory.
- if you have a short-lived program which allocates memory, does its work, and exits, there may be **no need to deallocate at all.**
  - for example, a program that reads some text, parses it into a tree, does some processing on that tree, and spits out different text.
    - what could I be talking about? :^)
- this idea can be applied at smaller scales *within* programs by using **arena allocation:** tying heap allocation *to the stack* by deallocating a group of objects when a function returns.
- buuuuut obviously this isn't a general solution.

Manual memory management

- since the programmer decides when object lifetimes **begin...**
- let's also have them decide when their lifetimes **end!**
  - hahaha what could possibly go wrong?

```
// C++
void main() {
 auto a = new A();
 a->method();
 delete a;
 a->method();
}
```

easy peasy!

oh fu----

this is not a solution to the problem.

we've just pushed it somewhere else.





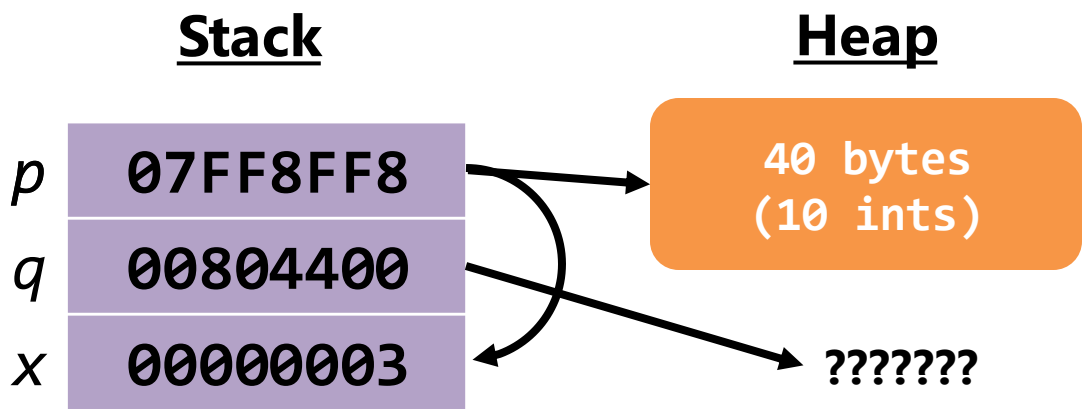
humans *suck* at knowing when is the right time to deallocate an object. **what makes it hard?**

It's those dang arrows (slightly animated)

- lots of languages make it **really, really easy** to create arrows…
  - but they offer little or no help in ensuring that those arrows are **valid** (that is, they point to a live object)

```
// C
int* p = malloc(40);
int* q = p + 256;
*q = 5000; // UB
free(p);
*p = 10;    // UB
int x = 3;
p = &x;
free(p);    // UB
```

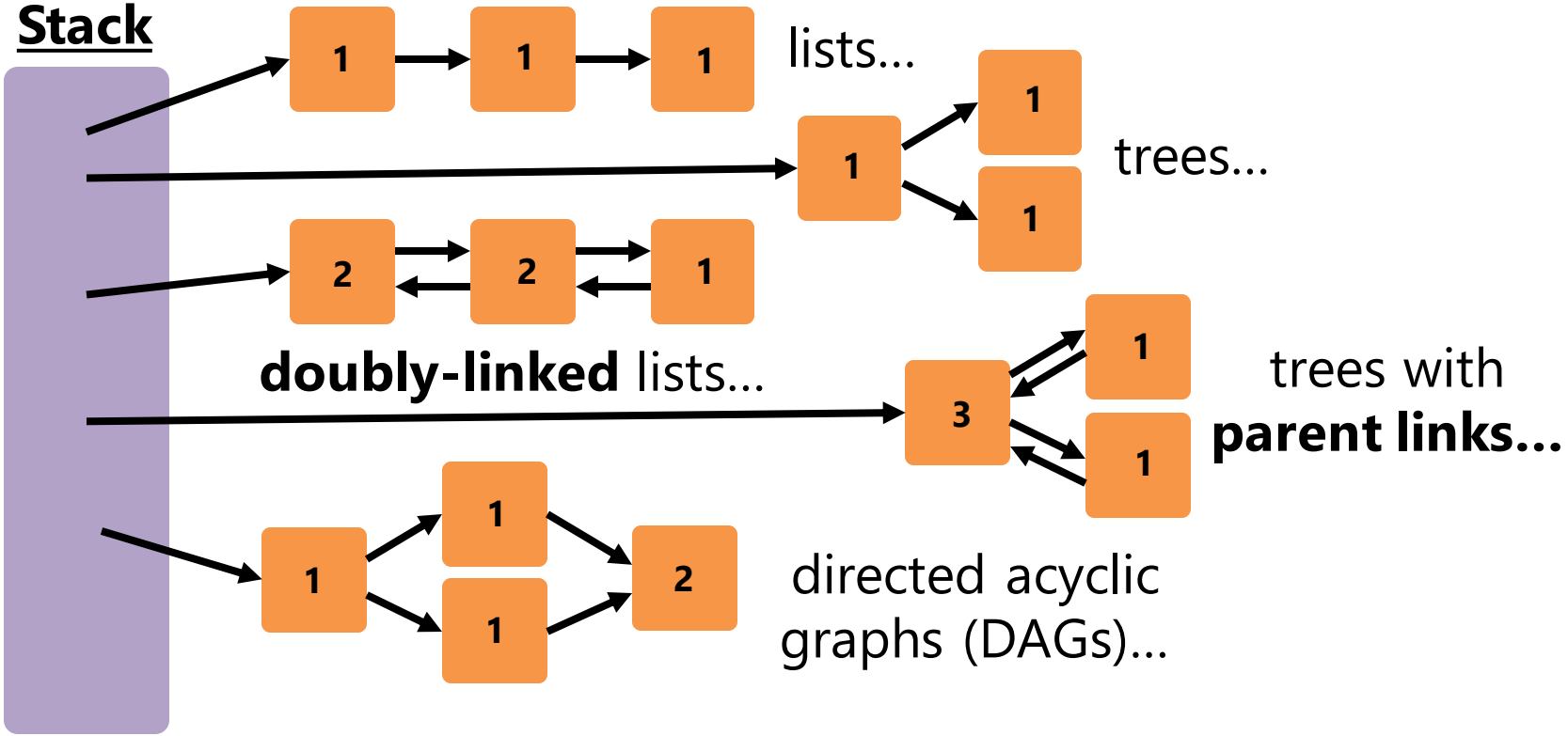
(UB = Undefined Behavior)



it gets worse.

Aliasing

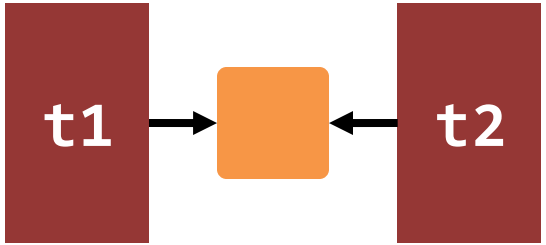
- this is when two or more arrows point to the **same value.**
- it's very powerful: it lets you create any directed graph (digraph).



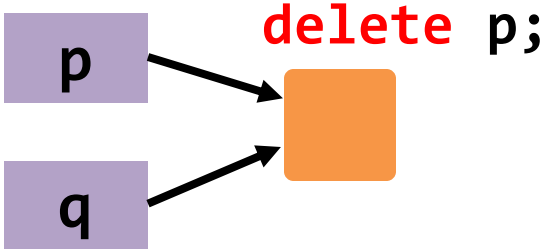
a node's **in-degree** is how many arrows point to it.

WITH GREAT POWER etc.

- **unrestricted** pointer aliasing can cause a lot of problems.



two threads of execution both accessing the **same object** at the **same time** can cause race conditions, deadlocks, and more.



if the programmer can deallocate objects, two pointers pointing at the same thing can lead to **dangling pointers.**

other sources of dangling/invalid pointers are **pointer arithmetic** and **pointers to stack values.**



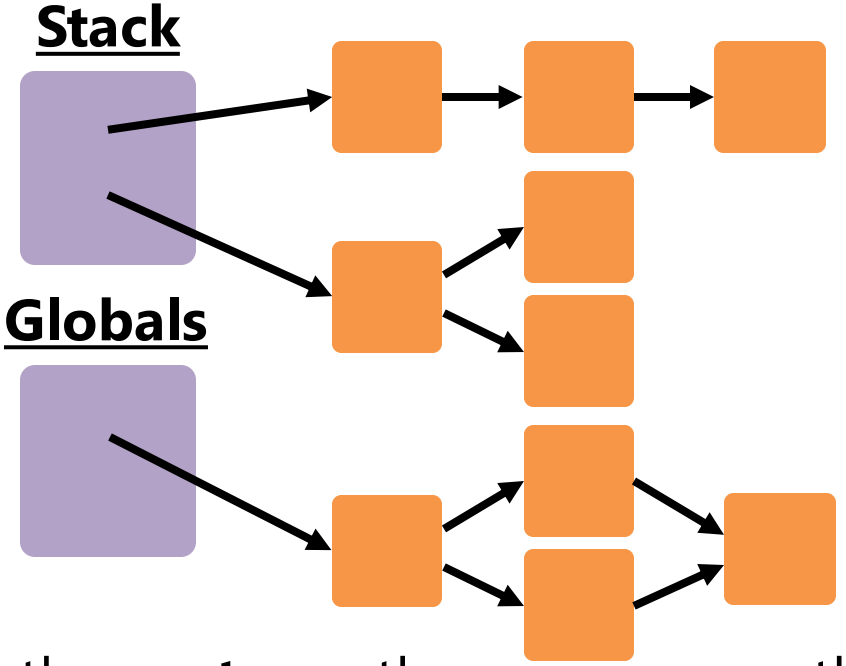
Automatic heap memory management

time check ≤ 70 min

Hard for humans, easy for computers (animated)

- **at a high level,** the problem is not that complicated.

objects and pointers form **graphs.**



reachable objects are pointed to – *directly* or *transitively* – from the roots.

any objects that become **unreachable** can never be used by the program again, and are therefore safe to deallocate.

the **roots** are the stack(s) and globals.

this gives us a simple* rule: **an object's lifetime ends when its in-degree reaches 0.**

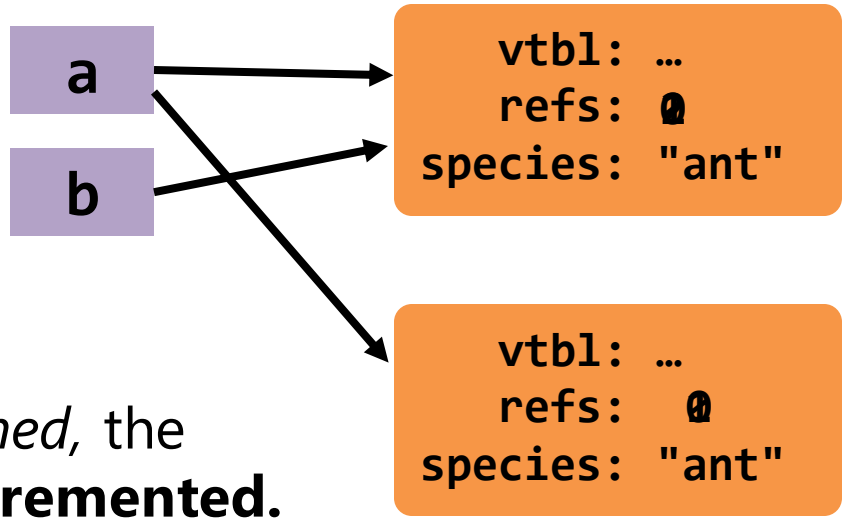
Counting the arrows by... counting them (animated)

- **reference counting (refcounting)** *explicitly* tracks the number of arrows pointing to an object at any given time.

whenever a reference to it is created, the count is **incremented.**

when an object is allocated, its refcount is 0.

```
Ant a = new Ant();
Ant b = a;
a = new Ant();
b = null;
a = null;
```



when a variable is *reassigned,* the previous object's count is **decremented.**

when the count reaches 0, the object is **deallocated.**

The compiler's job

- the compiler knows **when** the references are created and destroyed.
- so, around each assignment, it inserts those **'crements.**

```
Ant a = new Ant();
incRef(a);
Ant b = a;
incRef(b);
decRef(a);
a = new Ant();
incRef(a);
decRef(b);
b = null;
decRef(a);
a = null;
```

tricky details to consider:

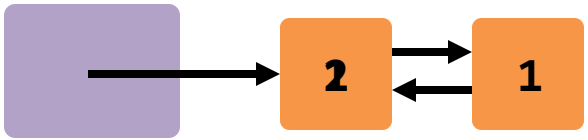
when a **local variable** goes out of scope, we need to decRef() it.

returning a reference or passing one as an **argument** will increment the refcount.

before an object is deallocated, **any references it has to other objects must be recursively decremented** first.

So what's the catch? (animated)

- "when an object's in-degree reaches 0" ***may never happen.***

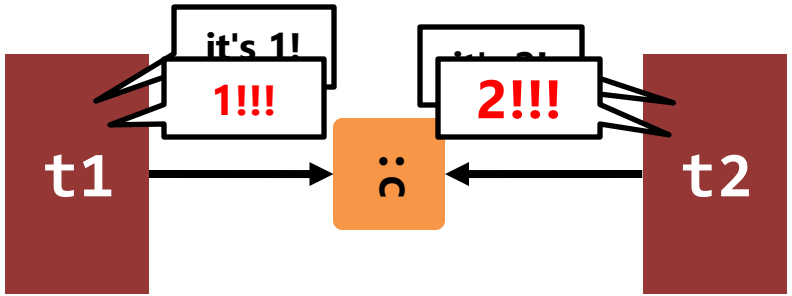


now what????????

these objects are **unreachable** and *should* be deallocated. **but they won't be.**

this is a **memory leak:** these objects take up space that can **never be deallocated.**

it's also bad for **multithreaded** programs, since the threads will fight over the refcount.



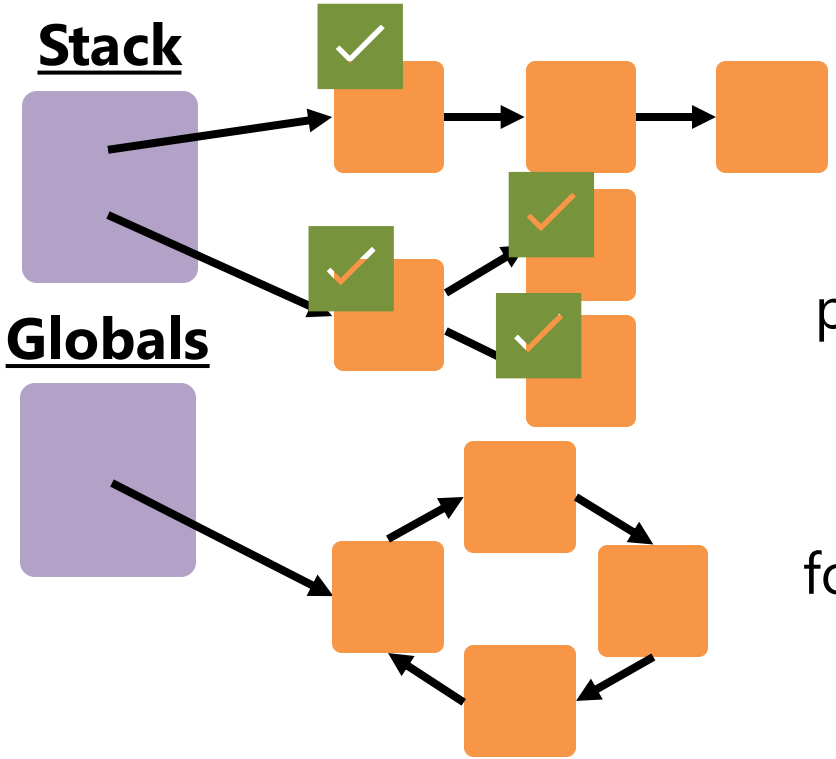
knowing the in-degree is *necessary* for automatic memory management, but it isn't *sufficient:* **it's not *all* you need to know.**

Tracing Garbage Collection

time check ≤ 90 min

The idea (animated)

- same ideas as before: roots, graph, reachability.



your program runs normally, with **no reference counting.** this is called the **mutation phase.**

periodically, your program is **paused** and the **collection phase** begins.

the **collector** starts at the roots, and follows every arrow it can find, marking each object it encounters as **live.**

when there are no more arrows to follow, the remaining objects are **garbage** and are swept away.

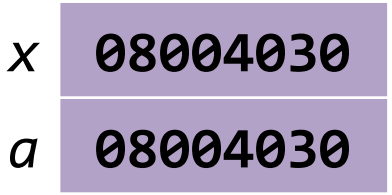
The correctness

- this algorithm is **mark-and-sweep** and is the basis of tracing GC.
- it works because an object is not garbage when its in-degree is 0...
  - it's garbage **when it is unreachable from the roots.**
- but wait, how does the collector *get to the dead objects?*
  - simple: **you are no longer the objects' owner. the collector is.**
- every time you allocate an object, **the collector remembers it.**
  - the collector has a list of *every object on the heap*.
  - in the mutation phase, it adds to this list when you **new.**
  - the collection phase, it uses this list to find disconnected objects.
- this works great for:
  - **cycles,** cause the collector doesn't care about in-degree.
  - **multithreading,** cause there are no ref counts to argue over, and the collector considers *all* threads' stacks as roots.

Concessions we make

- for the tracing algorithm to work well, there are some requirements.

```
let x = 0x08004030;
let a = new A();
```



what if this happened by chance? how can the collector know **a** is a pointer but **x** is not?

the **compiler** must produce information for **every global, local, and class field** saying whether or not something is a pointer.

```
int x = 0x08004030;
int *p = (int*)x;
```

type safety is crucial. being able to freely change the types of values will throw the collector way off.

pointer arithmetic is a no-go. nuh uh. no way.

What's the downside?

- well, the mark-and-sweep algorithm I showed is… too simple.
  - **it doesn't have great performance.**
- getting good performance means making it *way* more complex.
  - *concurrent* collectors do some/most of their work during the mutation phase, making the collection phase shorter.
  - *multithreaded* collectors extend that to multiple mutators.
  - *generational* collectors take advantage of the fact that most objects are deallocated very quickly after they're allocated, and focus more effort on newer objects.
  - *copying* collectors reduce the amount of memory needed by moving objects around.